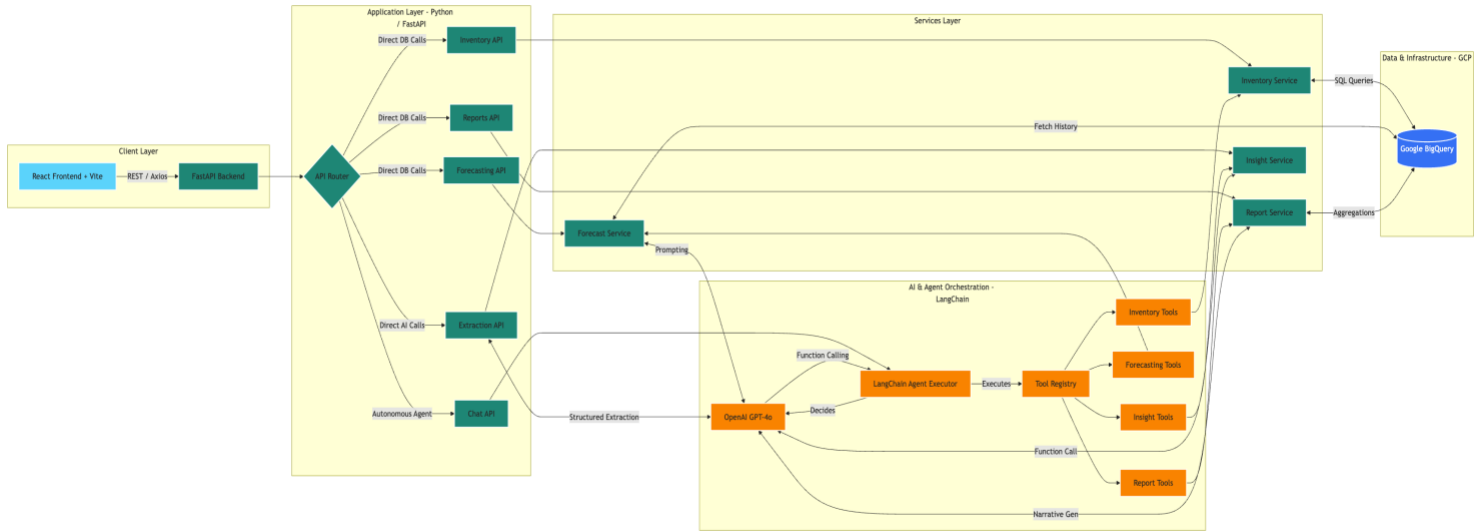


Report

AI Powered Retail Analytics Inventory Agent

Part 1: System Architecture Diagram



Part 2: End-to-End Flow & Technology Working

1. The Tech Stack & "Why"

Backend (Python/FastAPI): Chosen for its async capabilities, native Python support for data/AI libraries, and automatic OpenAPI documentation.

AI Engine (LangChain + OpenAI GPT-4o): LangChain provides the AgentExecutor and Tools framework. GPT-4o is used specifically because it natively supports Structured Outputs / Function Calling, which allows the LLM to reliably trigger Python functions (like running a SQL query) instead of just generating text.

Database (GCP BigQuery): A serverless, highly scalable data warehouse. We use it because retail sales data can grow to billions of rows. BigQuery can query terabytes in seconds. We also partition tables by date to save costs.

Frontend (React + Vite + Tailwind): Vite provides sub-second hot module replacement (HMR). Tailwind allows building a dark-mode, professional UI without writing custom CSS files. Recharts handles the data visualization for forecasts.

2. Step-by-Step Request Flow (Example: Chat Endpoint)

- i. **User Action:** User types in the React UI: "What items are out of stock, and generate a 14-day forecast for the worst one?"
- ii. **Network:** Axios sends a POST `/api/v1/chat/` request to the FastAPI backend.
- iii. **Agent Initialization:** FastAPI passes the prompt to the LangChain AgentExecutor.
- iv. **LLM Reasoning (Step 1):** GPT-4o reads the prompt and thinks: "I need to find out-of-stock items first. I will use the `query_inventory` tool with `status='out_of_stock'`."
- v. **Tool Execution (Step 1):** LangChain triggers the Python function `QueryInventoryTool._run()`.
- vi. **Database Hit:** The tool builds a SQL query (`SELECT * FROM inventory WHERE quantity_on_hand = 0`) and sends it to BigQuery via the `BigQueryService`.
- vii. **LLM Reasoning (Step 2):** GPT-4o reads the SQL results (e.g., "Wireless Earbuds Pro is out of stock"). It thinks: "Now I need to generate a forecast. I will use the `generate_forecast` tool for PROD-1001."
- viii. **AI Forecasting:** The forecast tool fetches 180 days of historical sales from BigQuery, formats it into a prompt, and sends it to GPT-4o to predict the next 14 days based on trends/seasonality.
- ix. **Final Synthesis:** GPT-4o combines the out-of-stock list and the forecast data into a coherent, human-readable response.
- x. **Response:** FastAPI returns the JSON to React, which renders it in the chat bubble, showing which tools were used.

Part 3: Detailed API Breakdown

1. Direct Endpoints (Non-Agent)

These endpoints bypass the AI agent and go straight to the services for speed and cost-efficiency.

- GET `/api/v1/inventory/summary`

How it works: Executes a GROUP BY SQL query in BigQuery joining inventory and products to calculate total SKUs, out-of-stock counts, and total inventory value.

- GET `/api/v1/inventory/alerts/low-stock`

How it works: Runs a filtered SQL query: `WHERE quantity_on_hand <= reorder_point`.

- POST `/api/v1/insights/extract`

How it works: Takes raw text (e.g., an email from a manager). Uses OpenAI's Function Calling feature. We define a strict JSON schema (products, metrics, sentiment) and force the LLM to extract data only into that schema. This prevents hallucinations.

2. Complex AI Endpoints

- POST `/api/v1/forecasting/generate`

How it works: Fetches daily aggregated historical sales. Passes the last 30 days of data, day-of-week patterns, and standard deviations to GPT-4o. The LLM outputs a 14-day JSON array of predictions, upper bounds, and lower bounds.

- POST /api/v1/reports/generate

How it works: Runs 3-4 separate BigQuery queries (e.g., category breakdowns, top sellers). Merges the resulting DataFrames into a giant JSON payload. Sends that payload to GPT-4o and asks it to "write an executive summary." Saves both the raw data and the AI summary to the reports table.

3. The Agent Endpoint

- POST /api/v1/chat/

How it works: The core LangChain loop. It has access to all the tools mentioned above. It maintains a session history (in-memory) so the user can ask follow-up questions ("What about the LA store instead?") and the agent remembers context.

Part 4: Formal Project Report

Architecture & Technical Design Report

1. Abstract

This project introduces an autonomous AI agent designed to modernize retail operations. By integrating Google Cloud BigQuery for petabyte-scale data warehousing, OpenAI's GPT-4o for cognitive reasoning, and LangChain for tool orchestration, the system moves beyond traditional dashboards. It provides a natural language interface where retail managers can query inventory levels, generate statistical and AI-driven demand forecasts, extract structured data from unstructured emails, and generate full narrative reports using a single chat interface.

2. System Architecture

The system follows a strict 3-tier architecture:

- Presentation Tier:** A React SPA utilizing Tailwind CSS for UI and Recharts for data visualization. Communicates via REST.
- Application Tier:** FastAPI running in a Docker container. Houses the Pydantic schemas for validation, the LangChain Agent core, and the Business Logic Services.
- Data Tier:** Google Cloud BigQuery. The schema is highly normalized, partitioned by date (for sales and forecasts), and clustered by product_id to minimize query costs and maximize speed.

3. Key Engineering Achievements

Reliable Structured Extraction: Instead of relying on standard LLM text generation (which is prone to formatting errors), the extraction service uses OpenAI's native Function Calling to enforce a strict Pydantic/JSON schema. This guarantees downstream code won't break when parsing AI outputs.

Hybrid Forecasting: The system doesn't just rely on AI. It implements standard statistical methods (Moving Average, Exponential Smoothing) as baselines, alongside the AI forecaster which can detect complex seasonality and weekend/weekday patterns.

Dynamic Tool Calling: The LangChain agent is not hard-coded. It reads the user prompt and dynamically decides which SQL queries to run, in what order, and how to synthesize the results.

Cost Optimization: BigQuery tables are partitioned by DATE(sale_date) and clustered by product_id, reducing query scan sizes by up to 90% compared to non-partitioned tables.

4. Data Pipeline Flow

Ingestion: Historical data is seeded via Pandas DataFrames directly into BigQuery using the google-cloud-bigquery PyArrow loader.

Storage: Raw sales are partitioned daily. AI-generated insights and reports are stored as JSON strings in dedicated tables for auditability.

Processing: On-demand. When a user requests a forecast, the service queries 180 days of partitioned data, aggregates it in BigQuery SQL, and transfers the aggregated result to the LLM.

5. Deployment Strategy

Containerization: Built via a multi-stage Dockerfile (reduces image size by separating build dependencies from runtime).

Infrastructure as Code (IaC): Terraform scripts define the GCP resources (BigQuery datasets, IAM roles for least-privilege access, Artifact Registry, Cloud Run).

Hosting: Deployed to GCP Cloud Run, a serverless compute platform that automatically scales from 0 to 1000 instances based on HTTP traffic, ensuring high availability with zero idle cost.

6. Future Scope

Real-time Streaming: Integrate Google Cloud Pub/Sub to process POS (Point of Sale) data in real-time, triggering automatic reorder alerts.

Multi-modal AI: Allow users to upload photos of store shelves, using GPT-4 Vision to detect empty shelves and automatically log out-of-stock events.

Vector Search: Embed product catalogs using OpenAI embeddings and store in BigQuery Vector Search to allow semantic product searches ("Find me alternatives to this out-of-stock item").